



DAY 4

C Basics — Writing, Compiling, Debugging

Complete Reference Guide with Theory, History, and In-Depth Concepts

*"C is the closest you can get to the hardware without writing
assembly,
and in embedded systems, that's exactly where you need to be."*

Table of Contents

1. Historical Background of C
2. Why C Rules Embedded Systems
3. The C Programming Environment
4. Writing C Programs (Deep Dive)
5. The GCC Compiler — Internals & Flags
6. The 4-Stage Compilation Process (Exploded View)
7. Debugging with GDB — Beyond Basics
8. C Data Types — Memory Layout & Alignment
9. The sizeof() Operator — Hidden Details
10. Format Specifiers & Type Safety
11. Hands-On Practice (With Explanations)
12. Advanced Research Topics
13. Common Pitfalls & Best Practices

1. Historical Background of C

Origin Story

- **Developed by:** Dennis Ritchie at Bell Labs (1972)
- **Predecessors:**
 - ALGOL (gave structure)
 - BCPL (gave brevity)
 - B (gave minimalism)
- **Why created:** To write the UNIX operating system (previously written in assembly)
- **Standardization:**
 - K&R C (1978) — unofficial standard
 - ANSI C (C89/C90) — first official standard
 - C99 — added `stdint.h`, `inline`, `long long`
 - C11 — added multi-threading, atomic operations
 - C17 — bug fixes, no new features (current standard)

Design Philosophy

"C is a language that gives the programmer maximum control with minimum restrictions."

- Trusts the programmer (no automatic bounds checking)
- Maps closely to assembly language
- Minimal runtime overhead
- Portable across architectures (with careful coding)

2. Why C Rules Embedded Systems

Technical Reasons

Feature	Why It Matters in Embedded
Direct Memory Access	Pointers allow reading/writing hardware registers
Bit Manipulation	Bitwise operators (&, , ^, <<, >>) control individual pins
No Garbage Collector	Predictable execution time (critical for real-time systems)
Inline Assembly	Can insert architecture-specific instructions
Small Footprint	Minimal runtime libraries (~20KB for bare-metal)
Interrupt Handlers	Can write ISRs (Interrupt Service Routines) directly

Industry Stats

- 90%+ of embedded firmware is written in C
- Linux kernel: ~95% C
- RTOS (FreeRTOS, Zephyr): 100% C

Conceptual Bridge

```
High-Level Language (Python/Java)
  ↓
[Abstraction Gap]
  ↓
C Language ← You are here
  ↓
Assembly
  ↓
Machine Code
  ↓
Hardware (Registers, Memory, Peripherals)
```

C fills the gap between human-readable logic and hardware control.

3. The C Programming Environment

Required Tools (Toolchain)

[Editor] → [Compiler] → [Assembler] → [Linker] → [Debugger]

Tool	Purpose	Example
Editor	Write source code	Vim, VS Code, Nano
Compiler	Translate C → Assembly	GCC, Clang, IAR, ARMCC
Assembler	Assembly → Object code	as (GNU assembler)
Linker	Combine objects + libraries	ld (GNU linker)
Debugger	Runtime inspection	GDB, LLDB
Make	Build automation	GNU Make, CMake

Directory Structure (Recommended)

```
~/embedded_projects/
├── c_programs/
│   ├── hello/
│   │   ├── hello.c
│   │   ├── hello.o
│   │   └── hello (executable)
│   ├── sizes/
│   │   ├── sizes.c
│   │   └── sizes
│   └── debug_demo/
│       ├── buggy.c
│       └── buggy (with debug symbols)
├── linux_scripts/
└── projects/
```

4. Writing C Programs (Deep Dive)

Anatomy of a C Program

```
// 1. PREPROCESSOR DIRECTIVES
#include <stdio.h>    // Standard I/O library
#include <stdint.h>   // Fixed-width integers (uint8_t, etc.)

// 2. MACROS
#define PI 3.14159
#define SQUARE(x) ((x) * (x)) // Macro with side-effect caution!

// 3. GLOBAL VARIABLES (Avoid in embedded unless necessary)
int global_counter = 0;

// 4. FUNCTION DECLARATIONS (Prototypes)
int add(int a, int b);

// 5. MAIN FUNCTION – Program entry point
int main(void) {
    // Local variables (stack-allocated)
    int result = add(5, 3);

    // Function call
    printf("Result: %d\n", result);

    // Return to OS
    return 0;
}

// 6. FUNCTION DEFINITIONS
int add(int a, int b) {
    return a + b;
}
```

Memory Segments in a C Program

Segment	Content	Storage
Text	Machine code (instructions)	Read-only
Data	Initialized globals/statics	Read-write
BSS	Uninitialized globals/statics (zeroed at startup)	Read-write
Heap	Dynamic memory (malloc/free)	Grows upward
Stack	Local variables, function frames	Grows downward

5. The GCC Compiler — Internals & Flags

What is GCC?

- GNU Compiler Collection
- Supports: C, C++, Objective-C, Fortran, Ada, Go, D
- Cross-compilation: Can compile for ARM, AVR, RISC-V, etc.

Essential GCC Flags

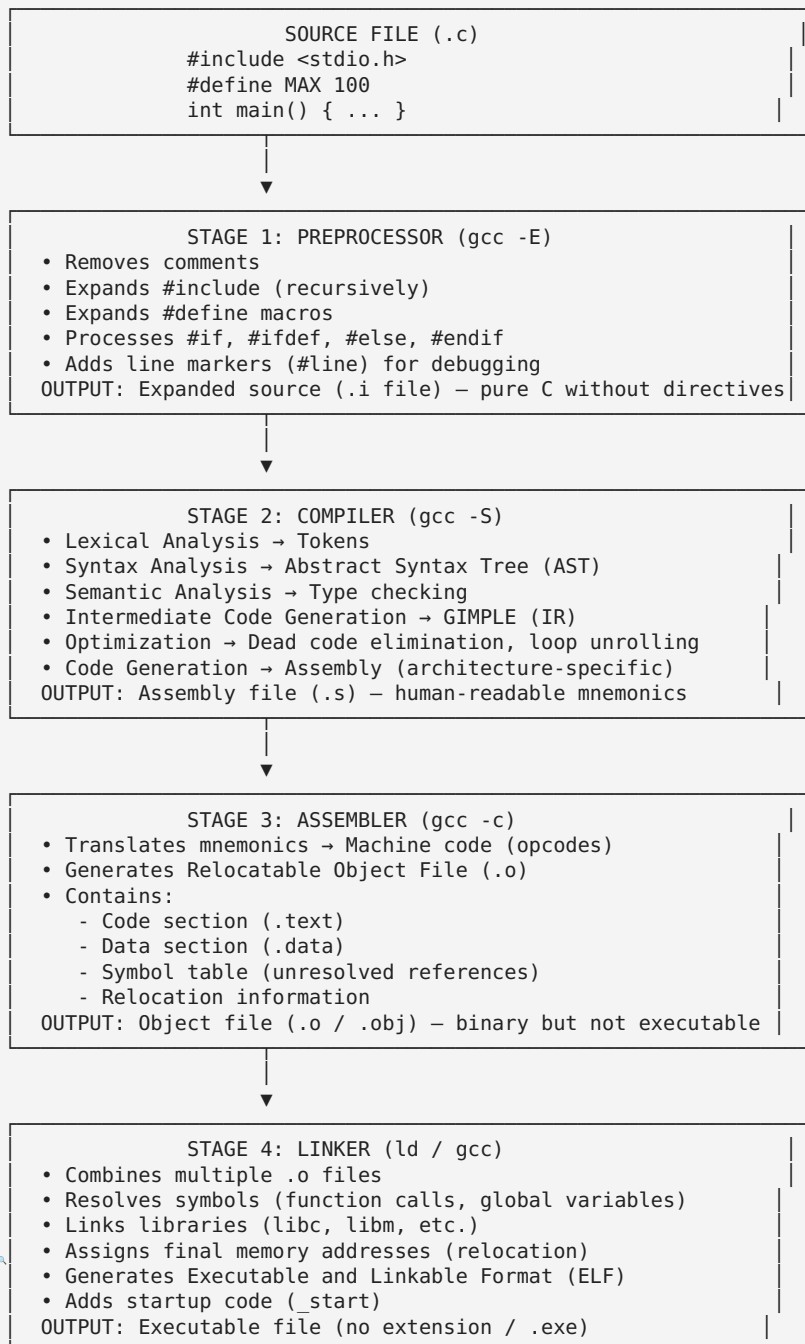
Flag	Purpose	Example
-o	Output filename	-o program
-Wall	All common warnings	Catch unused variable
-Wextra	Extra warnings	Catch sign comparison
-Werror	Treat warnings as errors	Forces fixes
-g	Debug symbols	For GDB
-O0	No optimization (default)	Fast compilation
-O1/-O2/-O3	Optimization levels	Smaller/faster code
-Os	Optimize for size	Embedded systems
-std=c99	Use C99 standard	-std=c11, -std=c17
-I	Include directory	-I./includes
-L	Library directory	-L./libs
-l	Link library	-lm (math library)

Compilation with Full Warnings (Production-Ready)

```
gcc -Wall -Wextra -Wpedantic -Werror -std=c17 -O2 -g program.c -o program
```

6. The 4-Stage Compilation Process (Exploded View)

Complete Flowchart



Deep Dive: Preprocessor Directives

```
#include <stdio.h> // Search in system paths (/usr/include)
#include "myheader.h" // Search in current directory first
```



```

#define DEBUG 1
#ifdef DEBUG
    printf("Debug mode\n");
#endif

// Macro with caution (evaluate twice!)
#define SQUARE(x) x * x    // BAD: SQUARE(3+2) → 3+2*3+2 = 11
#define SQUARE(x) ((x) * (x)) // GOOD

// Stringification
#define STR(x) #x
printf(STR>Hello)); // Prints "Hello"

// Token pasting
#define CONCAT(a, b) a##b
int var12 = 100;
printf("%d\n", CONCAT(var, 12)); // Prints 100

```

View Each Stage Manually

```

# 1. Preprocess only
gcc -E hello.c -o hello.i

# 2. Compile to assembly
gcc -S hello.i -o hello.s

# 3. Assemble to object
gcc -c hello.s -o hello.o

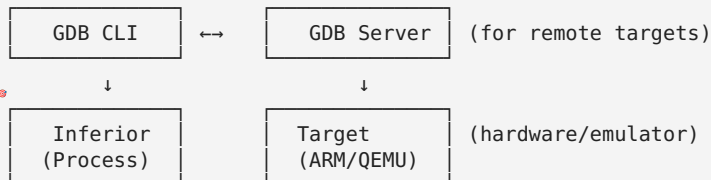
# 4. Link to executable
gcc hello.o -o hello

# View assembly code (Intel syntax)
objdump -d -M intel hello

```

7. Debugging with GDB — Beyond Basics

GDB Architecture



Essential GDB Commands

Command	Short	Purpose
break main	b main	Set breakpoint
break 10	b 10	Break at line 10
break func	b func	Break at function
run	r	Start execution
continue	c	Continue until next breakpoint
next	n	Step over (skip function calls)
step	s	Step into function
finish	fin	Run until current function returns
print var	p var	Print variable value
display var	disp	Auto-print on each stop
info locals	i lo	Show all local variables
info registers	i r	Show CPU registers
backtrace	bt	Show call stack
watch var	wa	Stop when var changes
set var=10	-	Change variable value
quit	q	Exit GDB

Debugging a Buggy Program

```
#include <stdio.h>

int divide(int a, int b) {
    return a / b; // Potential division by zero!
}
```

```
int main() {
    int x = 10, y = 0;
    int result = divide(x, y);
    printf("%d\n", result);
    return 0;
}
```

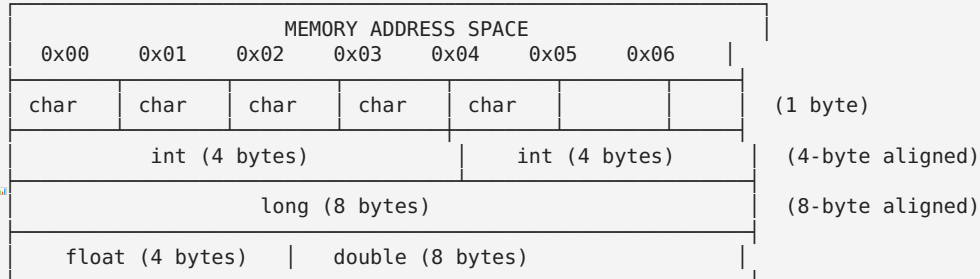
GDB Session:

```
gcc -g buggy.c -o buggy
gdb ./buggy

(gdb) break divide
(gdb) run
(gdb) print a      # Check parameters
(gdb) print b
(gdb) set var b = 2 # Fix value on the fly
(gdb) continue
# Program runs successfully!
```

8. C Data Types — Memory Layout & Alignment

Data Type Sizes (x86_64 Linux)



Complete Type Table with Limits

Type	Size (bytes)	Alignment	Range	Format
char	1	1	-128 to 127	%c / %hhd
unsigned char	1	1	0 to 255	%hhu
short	2	2	-32768 to 32767	%hd
unsigned short	2	2	0 to 65535	%hu
int	4	4	-2^{31} to $2^{31}-1$	%d
unsigned int	4	4	0 to $2^{32}-1$	%u
long	8	8	-2^{63} to $2^{63}-1$	%ld
unsigned long	8	8	0 to $2^{64}-1$	%lu
float	4	4	$\pm 1.18\text{e-}38$ to $\pm 3.4\text{e}38$	%f
double	8	8	$\pm 2.23\text{e-}308$ to $\pm 1.79\text{e}308$	%lf
long double	16	16	$\pm 3.36\text{e-}4932$ to $\pm 1.18\text{e}4932$	%Lf
void*	8	8	0 to $2^{64}-1$	%p
size_t	8	8	0 to $2^{64}-1$	%zu

Fixed-Width Types (from <stdint.h>)

```
#include <stdint.h>

int8_t  a;  // Exactly 8 bits signed
uint16_t b; // Exactly 16 bits unsigned
int32_t c;  // Exactly 32 bits signed
uint64_t d; // Exactly 64 bits unsigned
```

Portability Issue

```
// BAD: Size depends on platform
int my_array[100];

// GOOD: Fixed size
uint32_t my_array[100]; // Always 32 bits
```

Structure Padding & Alignment

```
struct example {
    char a;        // 1 byte → offset 0
    // 3 bytes padding for int alignment
    int b;         // 4 bytes → offset 4
    char c;        // 1 byte → offset 8
    // 3 bytes padding to align struct size to 4
};
// Total size: 12 bytes (not 6!)
```

To pack structures:

```
#pragma pack(push, 1)
struct packed {
    char a;
    int b;
    char c;
};
#pragma pack(pop)
// Size: 6 bytes
```

9. The sizeof() Operator — Hidden Details

What sizeof() Really Does

- Compile-time operator (not a function!)
- Evaluated by the compiler, not at runtime
- Returns type size_t (unsigned integer)

Usage Patterns

```
// 1. On type
printf("%zu\n", sizeof(int));           // 4

// 2. On variable
int x;
printf("%zu\n", sizeof(x));             // 4

// 3. On expression (no execution!)
int y = 5;
printf("%zu\n", sizeof(y + 10));        // 4 (y+10 is int)
// y is NOT evaluated — no change

// 4. On array
int arr[20];
printf("%zu\n", sizeof(arr));            // 80 (20 * 4)
printf("%zu\n", sizeof(arr[0]));         // 4
printf("%zu\n", sizeof(arr) / sizeof(arr[0])); // 20 elements

// 5. On string literal
printf("%zu\n", sizeof("Hello"));        // 6 (includes '\0')
printf("%zu\n", strlen("Hello"));        // 5 (no null)

// 6. On pointer
int *ptr = arr;
printf("%zu\n", sizeof(ptr));            // 8 (pointer size)
```

Important: sizeof vs strlen

```
char str[] = "Embedded";
printf("%zu\n", sizeof(str));           // 9 (including '\0')
printf("%zu\n", strlen(str));           // 8 (no null)
```

10. Format Specifiers & Type Safety

Complete Format Specifier Table

Specifier	Used For	Example
%d	signed int	printf("%d", -10);
%i	signed int (same as %d)	printf("%i", 10);
%u	unsigned int	printf("%u", 42);
%o	octal (unsigned)	printf("%o", 8); → "10"
%x	hex lowercase	printf("%x", 255); → "ff"
%X	hex uppercase	printf("%X", 255); → "FF"
%c	character	printf("%c", 'A');
%s	string	printf("%s", "Hello");
%f	float / double	printf("%.2f", 3.14159); → "3.14"
%e	scientific notation	printf("%e", 123.45); → "1.234500e+02"
%g	shorter of %f/%e	printf("%g", 123.45);
%p	pointer address	printf("%p", &x);
%zu	size_t	printf("%zu", sizeof(int));
%ld	long int	printf("%ld", 100000L);
%lld	long long int	printf("%lld", 100000000000LL);

Common Format Mismatches (Undefined Behavior!)

```
int x = 65;
printf("%c", x); // OK: prints 'A'
printf("%d", x); // OK: prints 65

float f = 3.14;
printf("%f", f); // OK
printf("%d", f); // WRONG: prints garbage!

long l = 100L;
printf("%ld", l); // OK
printf("%d", l); // WRONG on 64-bit (truncation)
```

Good Practice

```
// Cast to avoid warnings
printf("%d", (int)my_long_value);
```

11. Hands-On Practice (With Explanations)

Setup Directory Structure

```
mkdir -p ~/embedded_projects/c_programs
cd ~/embedded_projects/c_programs
```

Experiment 1: Hello World with All Flags

File: **hello.c**

```
#include <stdio.h>

int main(void) {
    printf("Hello, Embedded World!\n");
    return 0;
}
```

Commands:

```
gcc -Wall -Wextra -Wpedantic -std=c17 -O2 -g hello.c -o hello
./hello
objdump -d ./hello | less # View disassembly
```

Experiment 2: Data Type Sizes

File: **sizes.c**

```
#include <stdio.h>
#include <stdint.h>
#include <stdlib.h> // for size_t

int main() {
    printf("=== Basic Types ===\n");
    printf("char: %2zu bytes\n", sizeof(char));
    printf("short: %2zu bytes\n", sizeof(short));
    printf("int: %2zu bytes\n", sizeof(int));
    printf("long: %2zu bytes\n", sizeof(long));
    printf("long long: %2zu bytes\n", sizeof(long long));

    printf("\n=== Unsigned ===\n");
    printf("unsigned char: %2zu bytes\n", sizeof(unsigned char));
    printf("unsigned int: %2zu bytes\n", sizeof(unsigned int));

    printf("\n=== Floating ===\n");
    printf("float: %2zu bytes\n", sizeof(float));
    printf("double: %2zu bytes\n", sizeof(double));
    printf("long double: %2zu bytes\n", sizeof(long double));
}
```



```

printf("\n=== Fixed-width (stdint.h) ===\n");
printf("int8_t:      %2zu bytes\n", sizeof(int8_t));
printf("uint16_t:     %2zu bytes\n", sizeof(uint16_t));
printf("int32_t:      %2zu bytes\n", sizeof(int32_t));
printf("uint64_t:     %2zu bytes\n", sizeof(uint64_t));

printf("\n=== Pointers ===\n");
printf("void*:        %2zu bytes\n", sizeof(void*));
printf("char*:        %2zu bytes\n", sizeof(char*));

printf("\n=== Others ===\n");
printf("size_t:        %2zu bytes\n", sizeof(size_t));

return 0;
}

```

Compile & Run:

```

gcc -Wall -Wextra sizes.c -o sizes
./sizes

```

Experiment 3: Variable Manipulation

File: vars.c

```

#include <stdio.h>

int main() {
    // Integer types
    int a = 10;
    unsigned int b = 20;
    char c = 'A';
    float f = 3.14159f;
    double d = 3.14159265359;

    printf("Initial values:\n");
    printf("a = %d, b = %u, c = %c, f = %.5f, d = %.11f\n",
        a, b, c, f, d);

    // Modify
    a = a * 2;
    b += 5;
    c = c + 1;    // 'A' → 'B'
    f = f * 2.0f;
    d = d / 2.0;

    printf("After modification:\n");
    printf("a = %d, b = %u, c = %c, f = %.5f, d = %.11f\n",
        a, b, c, f, d);

    // Type conversion (implicit)
    int x = 10;
    float y = x / 3;        // 3 (integer division!)
    float z = x / 3.0f;     // 3.33333 (correct)
}

```

```
    printf("Integer division: %f\n", y);  
    printf("Float division:   %f\n", z);  
  
    return 0;  
}
```

Experiment 4: Using GDB

File: debug_demo.c

```
#include <stdio.h>  
  
int factorial(int n) {  
    if (n <= 1) return 1;  
    return n * factorial(n - 1);  
}  
  
int main() {  
    int nums[] = {5, 10, 0, 15};  
    int sum = 0;  
  
    for (int i = 0; i < 4; i++) {  
        int fact = factorial(nums[i]);  
        sum += fact;  
        printf("Factorial of %d = %d\n", nums[i], fact);  
    }  
  
    printf("Total sum of factorials: %d\n", sum);  
    return 0;  
}
```

GDB Session:

```
gcc -g debug_demo.c -o debug_demo  
gdb ./debug_demo  
  
(gdb) break factorial  
(gdb) break main  
(gdb) run  
(gdb) info locals  
(gdb) print nums  
(gdb) watch sum  
(gdb) continue # Watch stops when sum changes  
(gdb) backtrace  
(gdb) frame 0 # Show current frame  
(gdb) print n # Check recursion parameter  
(gdb) continue # Repeat  
(gdb) quit
```

12. Advanced Research Topics

Deep Dive Topics for Revision

1. Compilation Process in Detail

- **Lexical Analysis:** How the compiler breaks source into tokens
- **Parsing:** Building the Abstract Syntax Tree (AST)
- **Semantic Analysis:** Type checking, symbol resolution
- **Intermediate Representation:** GIMPLE, RTL (Register Transfer Language)
- **Optimization Passes:** Constant propagation, dead code elimination, loop unrolling
- **Code Generation:** Instruction selection, register allocation, scheduling

Command to see GCC IR:

```
gcc -fdump-tree-all hello.c # Dump all intermediate representations
```

2. Linking Process (Static vs Dynamic)

- **Static Linking:** Libraries copied into executable (larger, self-contained)
- **Dynamic Linking:** Shared libraries (.so) loaded at runtime (smaller, updates easier)
- **Symbol Resolution:** How linker finds function definitions
- **Relocation:** Assigning final memory addresses
- **Startup Code:** `_start` → `__libc_start_main` → `main`

View shared library dependencies:

```
ldd ./hello
```

3. Memory Layout of C Programs

- **Text Segment:** Executable instructions (read-only)
- **Data Segment:** Initialized static/global variables
- **BSS Segment:** Uninitialized static/global variables (zeroed)
- **Heap:** Dynamic allocation (malloc / free)
- **Stack:** Function call frames, local variables
- **Environment:** Command-line args, environment variables

Check memory map:

```
cat /proc/self/maps # For current process
```

4. Object File Format (ELF)

- **ELF Header:** Magic number, architecture, entry point
- **Program Headers:** Segment mapping for loader

- **Section Headers:** .text, .data, .bss, .rodata, .symtab, .strtab
- **Symbol Table:** Function and variable names with addresses

Inspect ELF:

```
readelf -h hello      # Header
readelf -S hello      # Sections
readelf -s hello      # Symbols
objdump -d hello      # Disassembly
```

5. ABI and Calling Conventions

- System V ABI (Linux x86_64)
- Register usage: RDI, RSI, RDX, RCX, R8, R9 for arguments
- Stack alignment: 16-byte alignment before call
- Caller-saved vs Callee-saved registers
- Return value: RAX register

13. Common Pitfalls & Best Practices

Common Mistakes

Pitfall	Example	Fix
Uninitialized variables	<code>int x; printf("%d", x);</code>	Always initialize
Buffer overflow	<code>char buf[10]; strcpy(buf, "too long");</code>	Use <code>strncpy</code> or check size
Integer overflow	<code>int x = INT_MAX + 1;</code>	Check limits
Division by zero	<code>int y = a / b; where b=0</code>	Always check divisor
Using <code>=</code> instead of <code>==</code>	<code>if (x = 5)</code>	Use <code>==</code> for comparison
Missing <code>break</code> in <code>switch</code>	<code>case 1: do_something();</code>	Add <code>break;</code>
Comparing signed/unsigned	<code>int a = -1; if (a > 0U)</code>	Cast or use same type
Incorrect format specifier	<code>printf("%d", 3.14);</code>	Use <code>%f</code>
Memory leak	<code>malloc()</code> without <code>free()</code>	Always free
Dangling pointer	<code>free(p); *p = 10;</code>	Set <code>p = NULL</code> after free

Best Practices for Embedded C

- Always use fixed-width types (`uint32_t`, `int16_t`, etc.)
- Initialize all variables — especially local ones
- Use `const` and `static` to limit scope and protect data
- Compile with `-Wall -Wextra -Werror` to catch issues early
- Use `volatile` for variables modified by interrupts/hardware
- Avoid dynamic allocation (`malloc/free`) in critical paths
- Use `assert()` for invariant checking (debug builds)
- Document bit-level operations with comments
- Use meaningful names — `timer_counter` not `tc`
- Keep functions small — one function does one thing

Day 4 — Completion Checklist

Theory Understanding

- ☐ History and philosophy of C language
- ☐ Why C is preferred for embedded systems
- ☐ The 4 stages of compilation (Preprocessor → Compiler → Assembler → Linker)
- ☐ Memory layout of C programs (Text, Data, BSS, Heap, Stack)
- ☐ Data types, sizes, and alignment
- ☐ Format specifiers and type safety
- ☐ Structure padding and packing
- ☐ GDB debugging basics
- ☐ Static vs dynamic linking
- ☐ ELF file format

Hands-On Skills

- ☐ Write and compile "Hello World" with warning flags
- ☐ Write a program to print all data type sizes
- ☐ Experiment with variables, assignments, and type conversions
- ☐ Use GDB to debug a program (breakpoints, watch, print, backtrace)
- ☐ View assembly code using objdump
- ☐ Preprocess, compile to assembly, assemble, and link manually
- ☐ Inspect ELF headers with readelf

Research Completed

- ☐ Compilation process (Preprocessor, Compiler, Assembler, Linker)
- ☐ Difference between sizeof() and strlen()
- ☐ Structure padding and alignment
- ☐ Static vs dynamic linking
- ☐ Calling conventions (ABI)

References & Further Reading

Books

- "The C Programming Language" (K&R)
- "Computer Systems: A Programmer's Perspective" (CSAPP)
- "Embedded C Coding Standard" (Barr Group)

Online

- GCC Manual: `man gcc`
- GDB Manual: `man gdb`
- ELF Specification (Linux Foundation)
- System V ABI Documentation

"C is the closest you can get to the hardware without writing assembly, and in embedded systems, that's exactly where you need to be."